

MATA54 - Estruturas de Dados e Algoritmos II

Hashing

Endereçamento Aberto - Realocação de Registros

George Lima e Flávio Assis

IC - Instituto de Computação

8 de abril de 2025

Motivação: diminuir cadeias de sondagens

Desempenho depende da posição dos registros

Exemplo (Hashing Duplo) $m = 11$.

Inserção: **18, 16, 27**

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	18
8:	
9:	27
10:	

Média de acessos: $5/3 = 1,67$

Motivação: diminuir cadeias de sondagens

Desempenho depende da posição dos registros

Exemplo (Hashing Duplo) $m = 11$.

Inserção: **18, 16, 27**

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	18
8:	
9:	27
10:	

Inserção: **18, 27, 16**

Média de acessos: $5/3 = 1,67$

Motivação: diminuir cadeias de sondagens

Desempenho depende da posição dos registros

Exemplo (Hashing Duplo) $m = 11$.

Inserção: **18, 16, 27**

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	18
8:	
9:	27
10:	

Inserção: **18, 27, 16**

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	27
6:	16
7:	18
8:	
9:	
10:	

Média de acessos: $5/3 = 1,67$

Motivação: diminuir cadeias de sondagens

Desempenho depende da posição dos registros

Exemplo (Hashing Duplo) $m = 11$.

Inserção: **18, 16, 27**

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	18
8:	
9:	27
10:	

Média de acessos: $5/3 = 1,67$

Inserção: **18, 27, 16**

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	27
6:	16
7:	18
8:	
9:	
10:	

Média de acessos: $4/3 = 1,33$

Realocação de registros

Critérios para realocação dos registros

Supondo que:

- ▶ cada registro é consultado várias vezes e
- ▶ todos os registros possuem igual probabilidade de serem consultados

Estratégia

Ao inserir registro com chave k , verificar se há vantagens em realocar registros no caminho da cadeia de sondagem de k .

Realocação de Registros

A cada inserção...

- ▶ Vários registros podem ser realocados, não apenas um

Ex.: Considere um arquivo após a inserção dos registros com chaves:

16, 18, 28, 31 ($m = 11$):

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	

Realocação de Registros

A cada inserção...

- ▶ Vários registros podem ser realocados, não apenas um

Ex.: Considere um arquivo após a inserção dos registros com chaves:

16, 18, 28, 31 ($m = 11$):

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	

Ao se inserir agora o 27, como realocar os registros?

Realocação de Registros

A cada inserção...

- ▶ Vários registros podem ser realocados, não apenas um

Ex.: Considere um arquivo após a inserção dos registros com chaves:

16, 18, 28, 31 ($m = 11$):

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	

Ao se inserir agora o 27, como realocar os registros?

Quais são as alternativas de realocação?

Realocação de Registros

A cada inserção...

- ▶ Vários registros podem ser realocados, não apenas um

Ex.: Considere um arquivo após a inserção dos registros com chaves:

16, 18, 28, 31 ($m = 11$):

Inserção do 27

Alternativa 1

End	Reg.
0:	27
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	

$$\text{Média: } \frac{1+1+1+1+4}{5} = 1,6$$

Realocação de Registros

A cada inserção...

- ▶ Vários registros podem ser realocados, não apenas um

Ex.: Considere um arquivo após a inserção dos registros com chaves:

16, 18, 28, 31 ($m = 11$):

Inserção do 27

Alternativa 1

End	Reg.
0:	27
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	

$$\text{Média: } \frac{1+1+1+1+4}{5} = 1,6$$

Alternativa 2

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	27
6:	16
7:	18
8:	28
9:	31
10:	

$$\text{Média: } \frac{2+1+2+1+1}{5} = 1,4$$

Estratégia geral

Algorithm 1: Algoritmo de busca e inserção de um registro r com chave $r.k$ numa tabela *hash* T de tamanho m usando duas funções *hash* h_1 e h_2 . Resolução de colisão por duplo *hashing* com realocação de registros.

```
1 duplo_hash_realloc( $r, T$ ):  
2    $i \leftarrow h_1(r.k)$   
3    $j \leftarrow h_2(r.k)$   
4    $c \leftarrow 1$   
5   while  $T[i].k \neq 0 \wedge T[i].k \neq r.k$  do                                /* busca  $r$  */  
6      $i \leftarrow (i + j) \bmod m$   
7      $c \leftarrow c + 1$   
8   if  $c = m$  then return  $-1$                                            /* tabela cheia */  
9   if  $T[i].k \neq k$  then                                                /*  $r$  não encontrado */  
10     $i \leftarrow \text{realloc}(k, T)$                                        /* reordena  $T$  para otimizar acessos */  
11     $(T[i].r, T[i].k) \leftarrow (r, k)$   
12  return  $i$ 
```

Possíveis estratégias para realocação de registros

Realocação só é realizada se houver um decréscimo do número de acessos total em relação a não considerar realocação

Métodos a serem estudados

- ▶ **Método de Brent.** Estratégia gulosa. Pode realocar até um registro. Método proposto por Richard P. Brent em 1973.
- ▶ **Método da árvore binária.** Estratégia baseada em avanço e retrocesso. Pode realocar vários registros. Método proposto por Gaston Gonnet e J. Ian Munro em 1979.

Estratégia

Ao se inserir um registro, escolhe-se um outro para se realocado, caso tal realocação seja vantajosa.

- ▶ Seja s o número de acessos totais acrescidos após a inclusão de um novo registro r sem realocação de registros (método duplo *hash*)
- ▶ Seja p o número de acessos totais acrescidos após a inclusão de um novo registro r com realocação de registros
 - ▶ Haverá $p - q$ acessos para o novo registro e q acessos acrescidos ao registro sendo realocado.
- ▶ **Só é vantagem realocar algum registro para inserir r se $p < s$!**

Método de Brent

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	

- ▶ Incluir 27 sem realocação **acrescentaria $s = 4$ acessos!**
- ▶ Testa-se mover algum registro, percorrendo a cadeia de sondagem de 27, acrescentando $p - q > 0$ acessos, para $p = 1, 2, \dots, s - 1$.
 - ▶ ($p = 1, q = 0$): casa 5 ocupada
 - ▶ ($p = 2, q = 0$): casa 7 ocupada
 - ▶ ($p = 2, q = 1$): casa 6 ocupada
 - ▶ ($p = 3, q = 0$): casa 9 ocupada
 - ▶ ($p = 3, q = 1$): **casa 8 vazia $\Rightarrow$ move-se 18 para incluir 27**

Método de Brent

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	

- ▶ Incluir 27 sem realocação **acrescentaria $s = 4$ acessos!**
- ▶ Testa-se mover algum registro, percorrendo a cadeia de sondagem de 27, acrescentando $p - q > 0$ acessos, para $p = 1, 2, \dots, s - 1$.
 - ▶ ($p = 1, q = 0$): casa 5 ocupada
 - ▶ ($p = 2, q = 0$): casa 7 ocupada
 - ▶ ($p = 2, q = 1$): casa 6 ocupada
 - ▶ ($p = 3, q = 0$): casa 9 ocupada
 - ▶ ($p = 3, q = 1$): **casa 8 vazia $\Rightarrow$ move-se 18 para incluir 27**

Método de Brent

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	

- ▶ Incluir 29 sem realocação **acrescentaria $s = 3$ acessos!**
- ▶ Testa-se mover algum registro, percorrendo a cadeia de sondagem de 29, acrescentando $p - q > 0$ acessos, para $p = 1, 2, \dots, s - 1$.
 - ▶ ($p = 1, q = 0$): casa 7 ocupada
 - ▶ ($p = 2, q = 0$): casa 9 ocupada
 - ▶ ($p = 2, q = 1$): casa 9 ocupada
 - ▶ ($p = 3, q = 0$): **casa 0 vazia $\Rightarrow$ inclui-se 29 na casa 0**

Método de Brent

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	

- ▶ Incluir 29 sem realocação **acrescentaria $s = 3$ acessos!**
- ▶ Testa-se mover algum registro, percorrendo a cadeia de sondagem de 29, acrescentando $p - q > 0$ acessos, para $p = 1, 2, \dots, s - 1$.
 - ▶ ($p = 1, q = 0$): casa 7 ocupada
 - ▶ ($p = 2, q = 0$): casa 9 ocupada
 - ▶ ($p = 2, q = 1$): casa 9 ocupada
 - ▶ ($p = 3, q = 0$): **casa 0 vazia $\Rightarrow$ inclui-se 29 na casa 0**

Método de Brent

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	

- ▶ Incluir 17 sem realocação **acrescentaria $s = 5$ acessos!**
- ▶ Testa-se mover algum registro, percorrendo a cadeia de sondagem de 17, acrescentando $p - q > 0$ acessos, para $p = 1, 2, \dots, s - 1$.
 - ▶ ($p = 1, q = 0$): casa 6 ocupada
 - ▶ ...
 - ▶ ($p = 3, q = 2$): **casa 10 vazia $\Rightarrow$ move-se 28 para casa 10 e inclui-se 17**

Método de Brent

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

- ▶ Incluir 17 sem realocação **acrescentaria $s = 5$ acessos!**
- ▶ Testa-se mover algum registro, percorrendo a cadeia de sondagem de 17, acrescentando $p - q > 0$ acessos, para $p = 1, 2, \dots, s - 1$.
 - ▶ ($p = 1, q = 0$): casa 6 ocupada
 - ▶ ...
 - ▶ ($p = 3, q = 2$): **casa 10 vazia $\Rightarrow$ move-se 28 para casa 10 e inclui-se 17**

Método de Brent

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

Configuração final

End	Reg.
0:	29
1:	26
2:	
3:	
4:	15
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

- ▶ Número médio de acessos com realocação pelo método de Brent: $19/9 \approx 2.11$
- ▶ Número médio de acessos sem realocação: $23/9 \approx 2.56$

Método de Brent

Algorithm 2: Método de Brent de realocação de registros numa tabela T para inclusão de um novo registro r .

```
1  realocaBrent( $r, T$ ):  
    /*  $p$  acessos a serem acrescentados após inclusão de  $r$ ,  $p - q$  acessos  
    para  $r$  e  $q$  para o registro sendo realocado */  
2   $p \leftarrow 1$   
3  while True do  
4      for  $q \leftarrow 0, \dots, p - 1$  do  
5           $i \leftarrow \{h_1(r.k) + (p - q - 1) \times h_2(r.k)\} \bmod m$   
6          if  $T[i].k = 0$  then /* não é necessário realocar registros */  
7              return  $i$   
8          else if  $q > 0$  then /* realocar  $T[i]$  acrescentando  $q$  acessos */  
9               $j \leftarrow \{i + q \times h_2(T[i].k)\} \bmod m$   
10             if  $T[j].k = 0$  then  
11                  $(T[j].r, T[j].k) \leftarrow (T[i].r, T[i].k)$  /* realoca  $T[i]$  */  
12                 return  $i$   
13      $p \leftarrow p + 1$ 
```

Método de Brent

Algorithm 3: Método de Brent de realocação de registros numa tabela T para inclusão de um novo registro r .

```
1  realocaBrent( $r, T$ ):  
    /*  $p$  acessos a serem acrescentados após inclusão de  $r$ ,  $p - q$  acessos  
    para  $r$  e  $q$  para o registro sendo realocado */  
2   $p \leftarrow 1$   
3   $q \leftarrow 0$   
4   $i \leftarrow h_1(r.k)$   
5  while  $T[i].k \neq 0$  do  
6       $p \leftarrow p + 1$   
7       $q \leftarrow 0$   
8       $i \leftarrow \{h_1(r.k) + (p - q - 1) \times h_2(r.k)\} \bmod m$   
9      while  $T[i].k \neq 0 \wedge q < p$  do  
10          $j \leftarrow \{i + q \times h_2(T[i].k)\} \bmod m$   
11         if  $T[j].k \neq 0$  then  
12             if  $T[j].k = 0$  then  
13                  $(T[j].r, T[j].k) \leftarrow (T[i].r, T[i].k)$  /* realoca  $T[j]$  */  
14                  $T[j].k \leftarrow 0$   
15              $q \leftarrow q + 1$   
16              $i \leftarrow \{h_1(r.k) + (p - q - 1) \times h_2(r.k)\} \bmod m$   
17          $p \leftarrow p + 1$   
18          $q \leftarrow 0$ 
```

Método da Árvore Binária

Os registros são acessados seguindo as cadeias de sondagem do registro que se quer inserir e dos demais registros acessados ao longo do processo.

Uma árvore binária é usada para determinar qual a melhor realocação de registros.

A árvore é montada nível a nível.

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	
6:	
7:	
8:	
9:	
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	
6:	
7:	
8:	
9:	
10:	

16
5 (-)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	
6:	
7:	
8:	
9:	
10:	

16
5 (-)

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	
8:	
9:	
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	
8:	
9:	
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	
8:	
9:	
10:	

18
7 (-)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	
8:	
9:	
10:	

18
7 (-)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	
8:	
9:	
10:	

18
7 (-)

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	18
8:	
9:	
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	18
8:	
9:	
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	18
8:	
9:	
10:	

28
6 (-)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	18
8:	
9:	
10:	

28
6 (-)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	
7:	18
8:	
9:	
10:	

28
6 (-)

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	
10:	

31
9 (-)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	
10:	

31
9 (-)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	
10:	

31
9 (-)

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	

27
5 (16)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

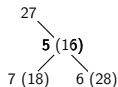
End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

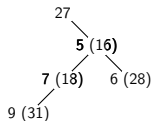
End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

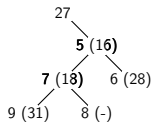
End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

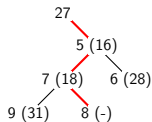
End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

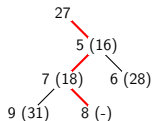
End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	



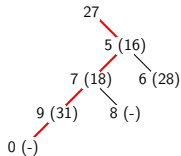
Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	



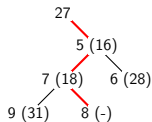
Por que não mover o 27 mais uma vez?



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

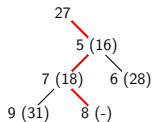
End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	18
8:	
9:	31
10:	



End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	

29
7 (27)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

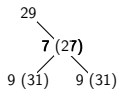
End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

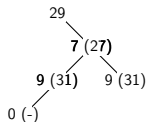
End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

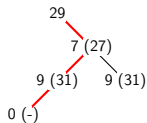
End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

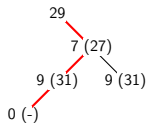
End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

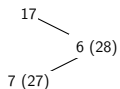
End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	

17
6 (28)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

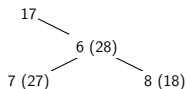
End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

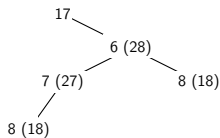
End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

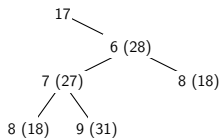
End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

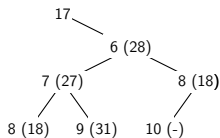
End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

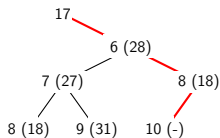
End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

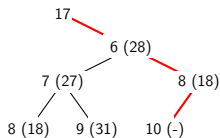
End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	28
7:	27
8:	18
9:	31
10:	



End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

26
4 (-)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

26
4 (-)

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	
2:	
3:	
4:	
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

26
4 (-)

End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

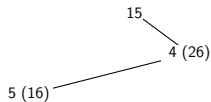
15
4 (26)

End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

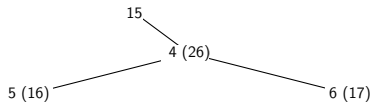
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

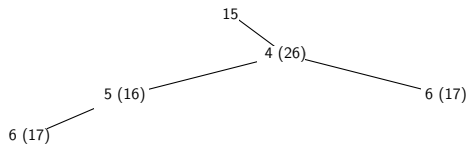
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

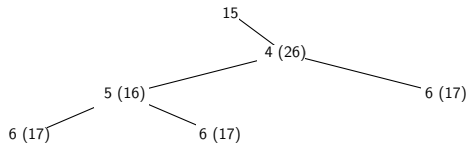
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

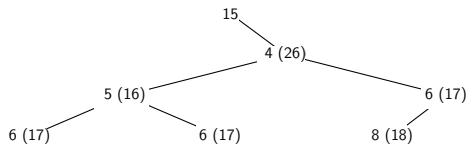
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

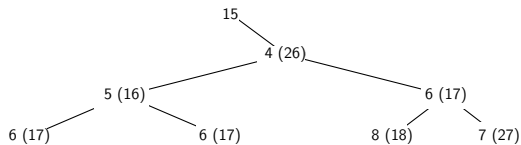
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

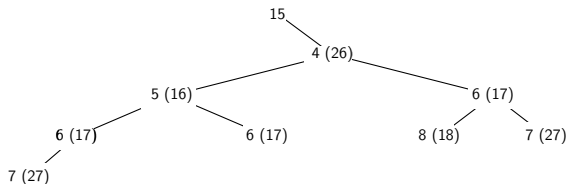
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

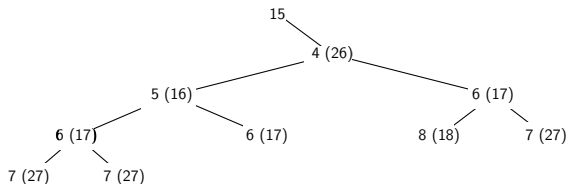
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

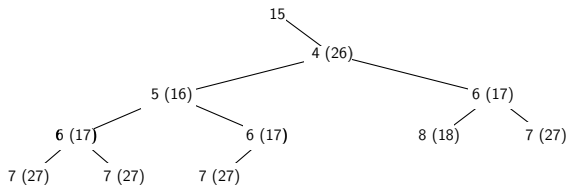
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

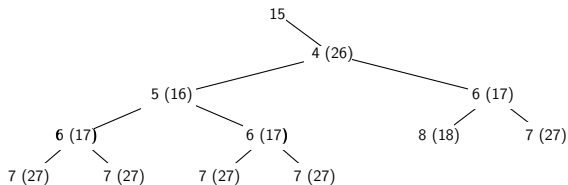
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

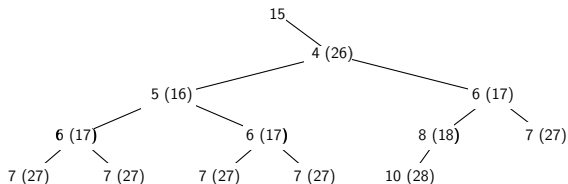
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

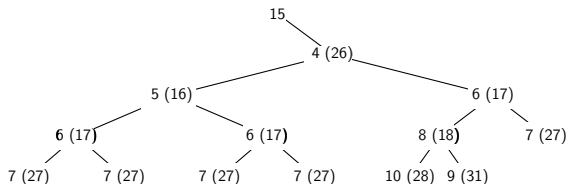
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

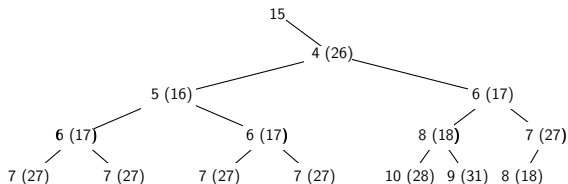
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

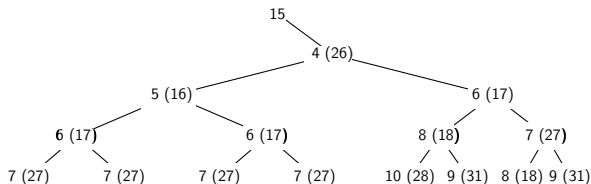
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

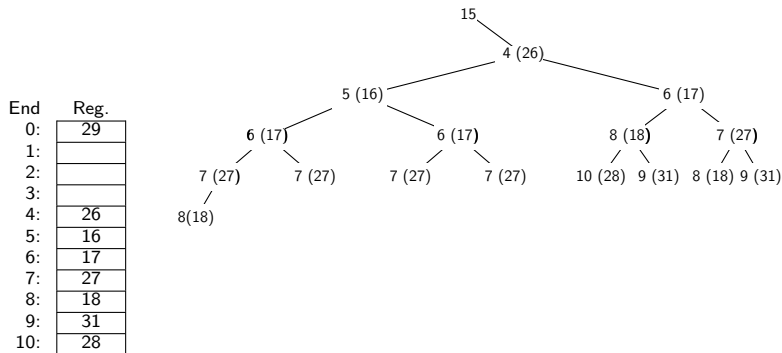
Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



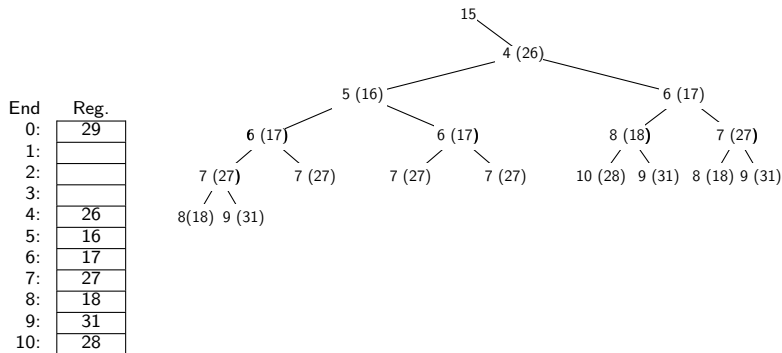
Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15



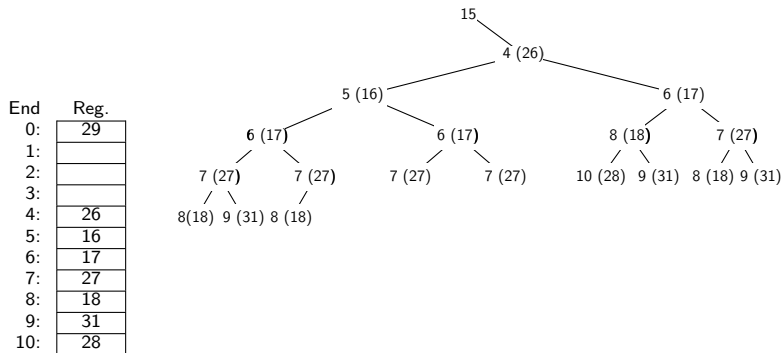
Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15



Árvore Binária

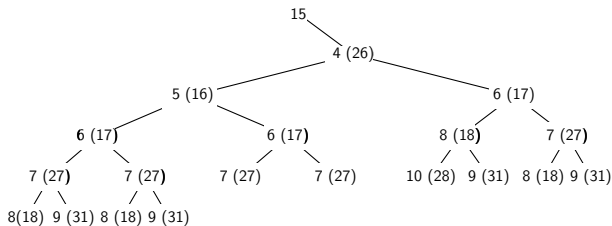
Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

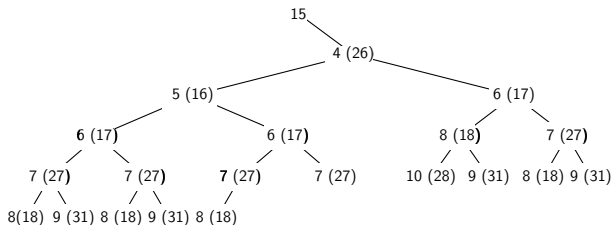
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

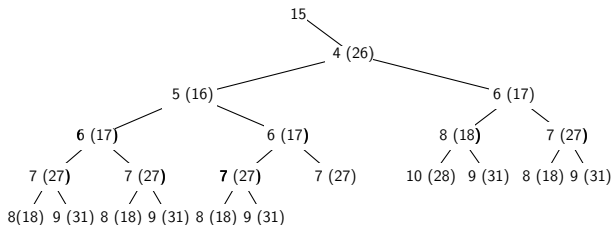
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

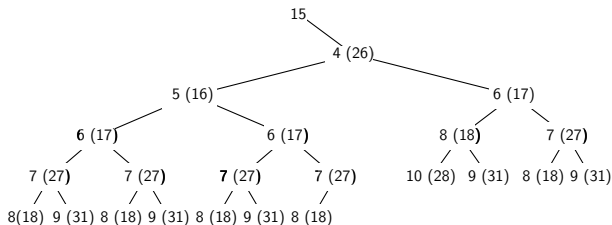
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

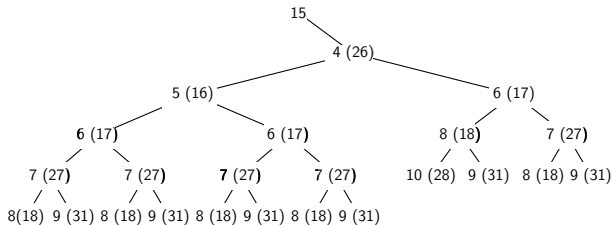
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

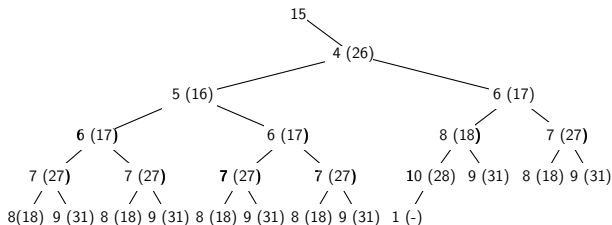
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

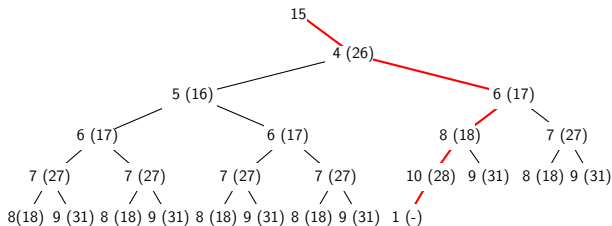
End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	
2:	
3:	
4:	26
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28



Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	26
2:	
3:	
4:	15
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

Árvore Binária

Exemplo: 16, 18, 28, 31, 27, 29, 17, 26, 15

End	Reg.
0:	29
1:	26
2:	
3:	
4:	15
5:	16
6:	17
7:	27
8:	18
9:	31
10:	28

Médias de acessos:

Árvore Binária: $\frac{23}{9} \approx 2,11$

Método de Brent: 2,11

Hashing Duplo (sem realocação): **2,56**

Sondagem Linear: **3,0**

Método da árvore binária: implementação

Modelagem para implementação

- ▶ A árvore A a ser construída é um vetor de tuplas. Seja $A[q] = (i, j)$ um nó da árvore, $q \geq 0$:
 - ▶ i é o endereço de T sondado quando e que deu origem ao nó $A[q]$.
 - ▶ Se $j \geq 0$, $A[j] = (i', j')$ é o nó tal que $T[i'].k$ é a chave usada na sondagem quando se gerou $A[q]$
 - ▶ Se $j < 0$, a chave usada na sondagem para a geração de $A[q]$ é a do registro sendo inserido $r.k$.
- ▶ Após construir a árvore, a partir do último nó criado, efetua-se as movimentações dos registros (caso necessário)
Seja $A[q] = (i, j)$ o último nó gerado para A . Sabe-se que $T[i] = 0$ (endereço i disponível)
 - ▶ Seja $A[j] = (i', j')$. Copia-se para $T[i]$ o registro armazenado em $T[i']$
 - ▶ Faz-se $q \leftarrow j'$ e, se $q \geq 0$, repete-se o passo anterior. Se $q < 0$, o endereço i

Método da árvore binária: implementação

```
1  ArvBinRealoc( $r, T$ ):
2       $q \leftarrow 0$ ;  $i \leftarrow h_1(r.k)$ ;  $j \leftarrow -1$            /* raiz da árvore  $A[0] = (i, -1)$  */
3      while  $T[i].k \neq 0$  do                               /* construção da árvore  $A$  */
4           $A[q] \leftarrow (i, j)$                           /* insere nó  $q$  na árvore */
5           $q \leftarrow q + 1$                                /* próximo nó a ser criado */
6           $p \leftarrow \lfloor (q - 1) / 2 \rfloor$              /* índice do pai do próximo nó a ser criado */
7          Seja  $A[p] = (i, j)$ 
8          if  $q \bmod 2 = 1$  then                             /*  $A[q]$  é filho esquerdo de  $A[p]$  */
9              if  $j = -1$  then  $k \leftarrow r.k$            /* próxima sondagem usa a chave de  $r$  */
10             else
11                 Seja  $A[j] = (i', j')$ 
12                  $k \leftarrow T[i'].k$                  /*  $A[j]$  indica a chave da próxima sondagem */
13             else                                         /*  $A[q]$  é filho direito de  $A[p]$  */
14                  $k \leftarrow T[i].k$                    /*  $A[p]$  indica a chave da próxima sondagem */
15                  $j \leftarrow p$                          /* para sondagens a partir de  $A[q]$  */
16              $i \leftarrow (i + h_2(k)) \bmod m$            /* endereço da próxima sondagem */
17             /* último nó  $(i, j)$  criado não foi inserido na árvore */
18         while  $j \neq -1$  do                               /* realocação de registros */
19             Seja  $A[j] = (i', j')$ 
20              $T[i'] \leftarrow T[i']$ 
21              $(i, j) \leftarrow A[j]$                      /* nó em nível superior na árvore */
22     return  $i$ 
```

- ▶ Conjetura-se que o comprimento médio das cadeias de sondagem é $O(\log n)$ [Gonnet, Baeza-Yates]. Este resultado é verificado em simulações.
- ▶ O custo de inserção pode ser muito alto, quando a tabela/arquivo está cheia(o).
- ▶ Pode ser uma alternativa para conjuntos de dados estáticos.

Exercícios

1. Considerando uma tabela de tamanho $m = 11$, e as funções h_1 e h_2 vistas em sala, insira os registros com as seguintes chaves numa tabela *hash*, nesta ordem

72, 47, 63, 15, 54, 16, 13, 34, 46, 17

Use duplo *hash* com e sem realocação de registros. Para realocação, use os métodos de Brent e da árvore binária. Mostre a configuração final da tabela para cada caso.

2. Considerando os métodos de Brent e da árvore binária e partindo da mesma configuração da tabela *hash*, forneça o valor de uma chave k tal que há um número distinto de realocações de registros.
3. Forneça a expressão do limite máximo para o número de sondagens realizadas ao se incluir um registro numa tabela *hash* de tamanho m com n registros, considerando:
 - 3.1 Método de Brent
 - 3.2 Método da árvore binária

- ▶ Realocação de registros diminui (em geral) número de médio acessos
- ▶ Maior esforço durante a inserção pode compensar operações de busca
- ▶ Método de Brent testa um subconjunto (de tamanho polinomial) de possibilidades comparado com método da árvore binária (de tamanho exponencial)
- ▶ Implementação da árvore pode ser eficiente usando vetores para representá-la
- ▶ Árvore binária pode crescer bastante, mas tal crescimento pode ser monitorado, e mesmo antecipado, pelo fator de carga